

深圳大学实验报告

课程名称： 计算机网络

实验项目名称： 实验 4 Socket 网络编程

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 王璐

报告人： 汉雨 学号： 2024040042 班级：

实验时间： 20260501

实验报告提交时间： 20260506

实验目的

1. 理解 TCP 与 UDP 在连接方式、传输特点和适用场景上的差别。
2. 掌握 Python 中 requests、socket、threading 等常用库在网络程序里的基本用法。
3. 通过 URL 请求、时间查询、文件传输和网络聊天室四个实验，把客户机/服务器程序的基本写法真正跑通。
4. 在写报告的过程中，不只是给出结果，还要说明代码为什么这样写、每一步做了什么、运行后看到了什么。

实验环境

1. 运行环境：Python 3.x。
2. 主要库：requests、socket、os、time、threading。
3. 测试方式：本实验主要在本机回环地址 127.0.0.1 上完成联调，URL 请求部分额外启用了本地测试网页，以便稳定拿到真实截图。
4. 文档说明：个人信息先保留待填写，正文中的截图、流程图和代码讲解图均围绕这次真实运行结果来整理。

实验内容与要求

实验 4 一共包括四项内容：URL 请求程序、系统时间查询、网络文件传输、网络聊天室。

其中系统时间查询和网络文件传输要求使用 TCP，网络聊天室要求使用 UDP。

按照实验指导，报告里不仅要有实现步骤，还应尽量把 核心代码、运行结果、现象解释 交代清楚。

参考样例报告的写法，下面的正文按“代码思路与编写”和“程序运行结果”两层来展开，并补入较多图示，尽量让阅读过程更直观。

实验步骤

1 URL 请求程序

要求：利用 Python 的 HTTP 库 Requests 实现一个简单程序，能够请求网页并存储为 HTML 文件，同时统计所请求网页的大小。

1.1 代码思路与编写

该程序的实现思路可以概括为四个环节：读入 URL、发送请求、保存网页、统计结果。附件中的 `url_download.py` 基本遵循了这一顺序，因此程序结构较为清晰。

首先，程序通过 `input()` 接收用户输入的目标地址，再调用 `requests.get()` 发起 HTTP 请求。由于返回的是响应对象，因此既可以获取实际访问到的 URL，也可以直接读取网页的原始内容。

其次，程序编写了 `saveHtml(file_name, file_content)` 函数，用于将网页内容按二进制方式写入本地 HTML 文件。采用二进制写入可以减少编码转换对页面内容造成的干扰。

最后，程序利用 `os.path.getsize()` 统计保存后文件的字节数，再通过 `os.getcwd()` 输出保存路径。至此，实验指导中要求展示的 URL、文件名、文件大小等信息都能够完整呈现。

URL Request Program Flow

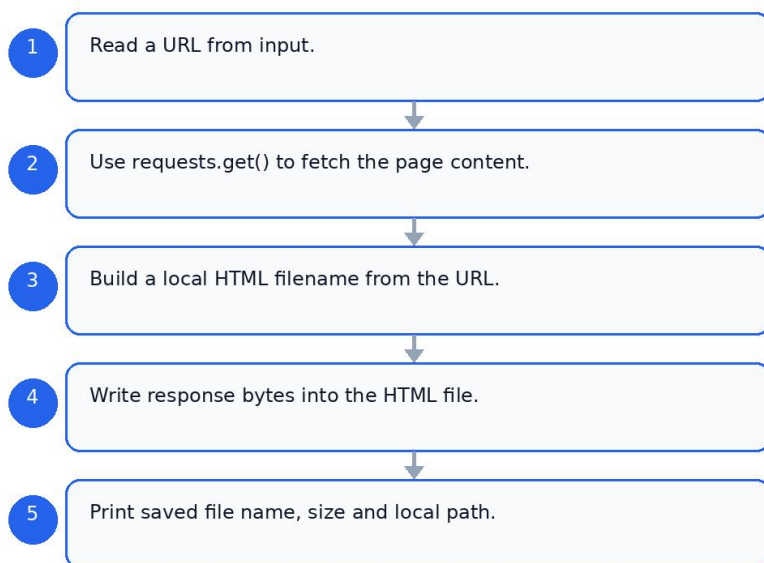


图 1.1.1 URL 请求程序的执行流程

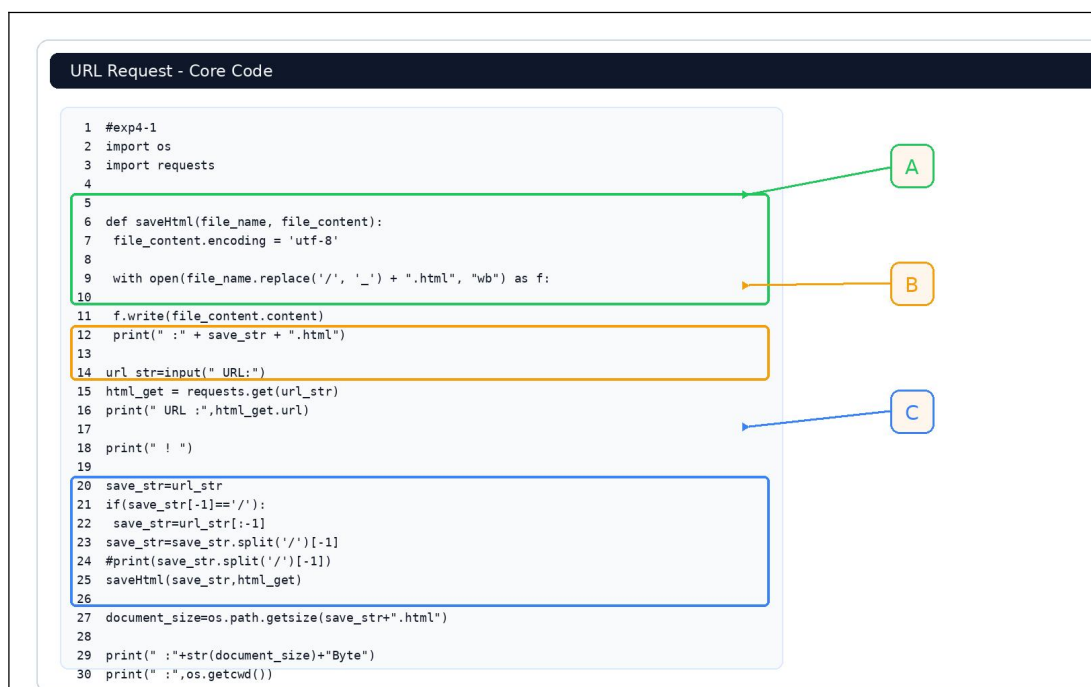


图 1.1.2 URL 请求程序核心代码及说明

图中 A：对应网页保存函数。该部分负责将响应对象中的内容写入本地 HTML 文件，是本实验实现网页落盘的关键。

图中 B：对应 URL 输入与请求发送部分。程序从用户处读取 URL 后，立即调用 `requests.get()` 发起访问。

图中 C：对应结果输出部分。该部分集中展示真实请求地址、保存文件名、文件大小以及保存路径。

1.2 程序运行结果

为了避免外部网络波动对实验结果造成影响，本次测试采用本地测试网页，并将 `http://127.0.0.1:18080/` 作为输入地址。实验结果表明，程序能够正确输出请求地址、保存文件名、文件大小以及保存路径。

同时，保存后的 HTML 文件已经实际生成，说明本实验不仅完成了请求动作，而且已经完成了本地文件写入。该现象与实验要求保持一致。



图 1.2.1 URL 请求程序运行结果

2 系统时间查询

要求：实现一个基于客户机/服务器的系统时间查询程序，传输层使用 TCP。客户端发送 `Time` 后，服务器返回当前系统时间；客户端发送 `Exit` 后，服务器返回 `Bye` 并结束连接。

2.1 代码思路与编写

该实验由服务端与客户端两部分组成，对应脚本分别为 `TCP_inquiry_server.py` 和 `TCP_inquiry_client.py`。二者均通过 `socket(AF_INET, SOCK_STREAM)` 创建 TCP 套接字。

客户端的处理流程较为直接：先输入服务器 IP 地址并完成连接，随后循环读取用户输入的指令，并将其发送至服务器。若发送的是 `Time.`，则等待服务器返回时间字符串；若发送的是 `Exit.`，则在接收到 `Bye.` 后主动关闭套接字。

服务端首先绑定 `0.0.0.0:12000` 并进入监听状态。一旦有客户端接入，即开始接收指令并进行分支处理：收到 `Time.` 时调用 `time.asctime()` 获取当前系统时间并回传；收到 `Exit.` 时返回 `Bye.` 并关闭连接。

从程序结构来看，该实验属于典型的 TCP 请求—响应式通信模型。其关键并不在于代码复杂度，而在于应用层命令字是否统一、通信顺序是否严格对应。

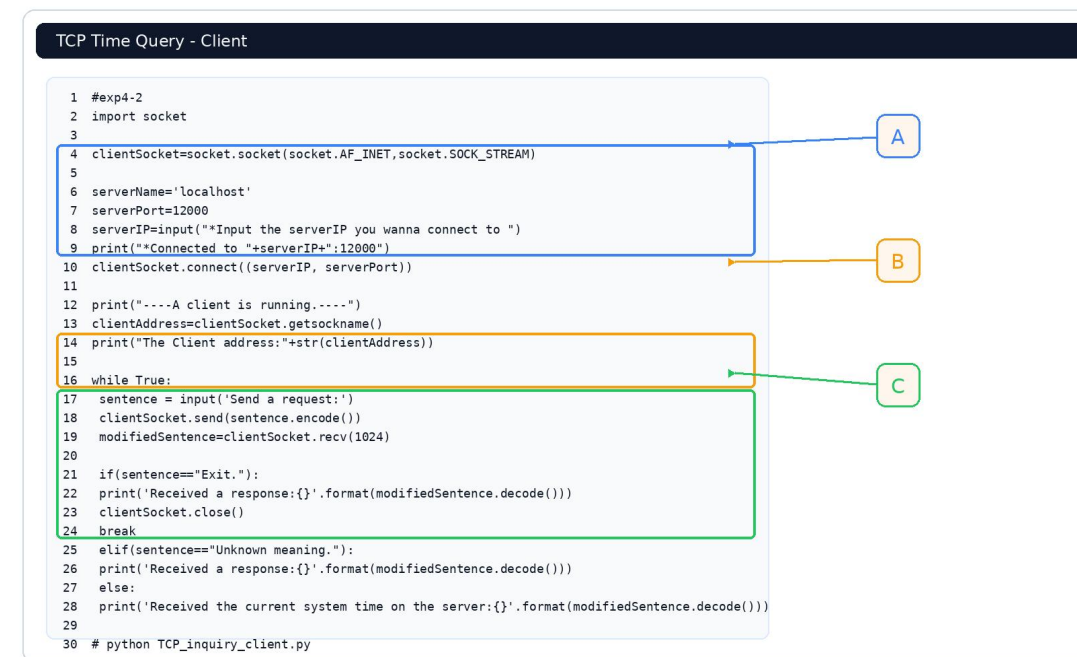


图 2.1.1 TCP 时间查询客户端核心代码

图中 A：对应客户端初始化与连接建立部分。程序在此创建 TCP 套接字，并根据输入的

服务器地址完成连接。

图中 B: 对应客户端发送请求部分。程序从键盘读取字符串后, 通过 `send()` 将请求发往服务器。

图中 C: 对应客户端响应处理部分。程序分别对 `Time.` 与 `Exit.` 的返回结果进行输出, 从而保证实验过程清晰可读。

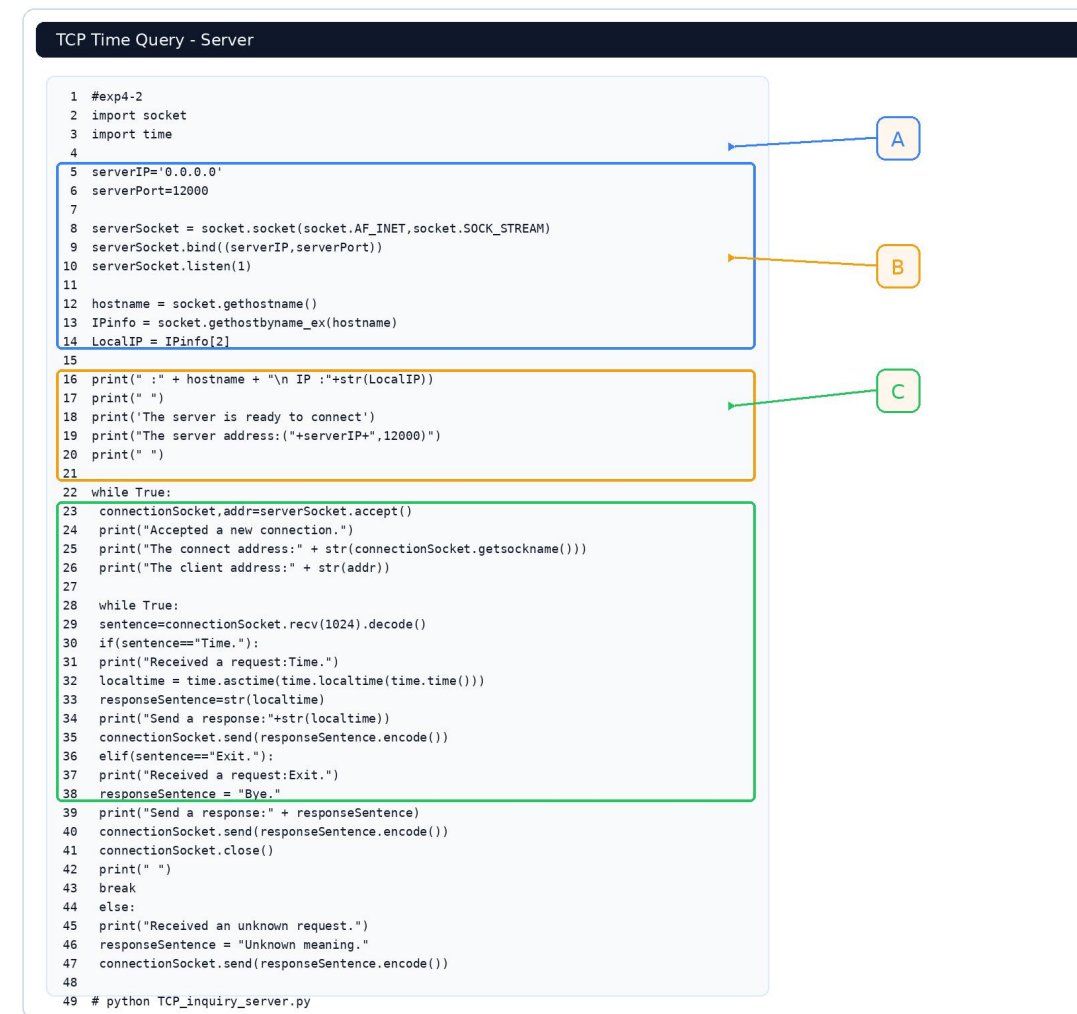


图 2.1.2 TCP 时间查询服务端核心代码

图中 A: 对应服务端创建、绑定与监听部分, 是整个时间查询服务启动的基础。

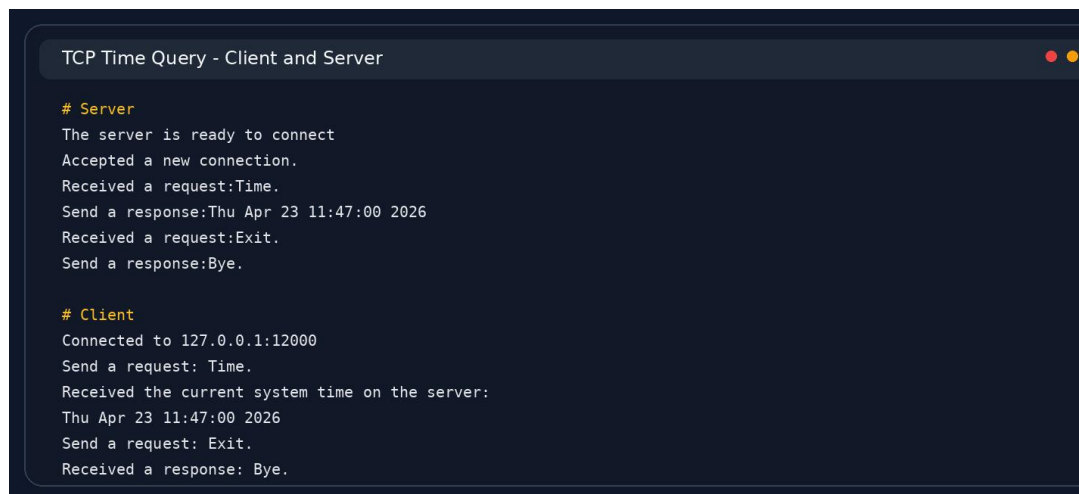
图中 B: 对应服务端信息输出部分, 用于显示当前主机名、可供连接的地址以及监听端口。

图中 C: 对应服务端请求处理循环。服务器在此根据收到的命令返回系统时间或 `Bye.`。

2.2 程序运行结果

测试时先启动服务端, 再启动客户端, 并依次输入 `Time.` 与 `Exit.`。实验结果表明, 客户端能够正确接收到服务器返回的当前时间, 之后又能够收到 `Bye.` 作为结束响应。

与此同时，服务端完整打印了“接收到何种请求”和“返回了何种结果”的过程，这说明客户端与服务端之间的命令语义已经完全对应，实验要求得到了满足。



```
TCP Time Query - Client and Server

# Server
The server is ready to connect
Accepted a new connection.
Received a request:Time.
Send a response:Thu Apr 23 11:47:00 2026
Received a request:Exit.
Send a response:Bye.

# Client
Connected to 127.0.0.1:12000
Send a request: Time.
Received the current system time on the server:
Thu Apr 23 11:47:00 2026
Send a request: Exit.
Received a response: Bye.
```

图 2.2.1 TCP 系统时间查询运行结果

3 网络文件传输

要求：实现一个基于客户机/服务器的网络文件传输程序，传输层使用 TCP。客户端先输入文件名，服务器收到后传输文件，客户端接收后重命名并保存到本地。

3.1 代码思路与编写

该实验同样由服务端和客户端组成，分别对应 `TCP_transfer_server.py` 与 `TCP_transfer_client.py`。服务端负责读取 `filehub` 目录中的目标文件，客户端负责提出请求并接收文件。

为了使客户端能够准确判断文件接收何时结束，程序采用了较为规范的处理方式：先发送文件总大小，再发送文件正文。客户端据此累计已接收字节数，当达到目标值时结束接收循环。

服务端在收到文件名之后，会先拼接出文件路径并读取文件总大小，随后进入循环分块读取文件，并将每一块内容通过 TCP 套接字发往客户端。

客户端则在接收到文件大小信息后，新建一个以 `received_` 为前缀的本地文件，并将后续收到的数据逐块写入。该设计既能避免覆盖原始文件，也便于实验后核对传输结果。

TCP File Transfer Flow

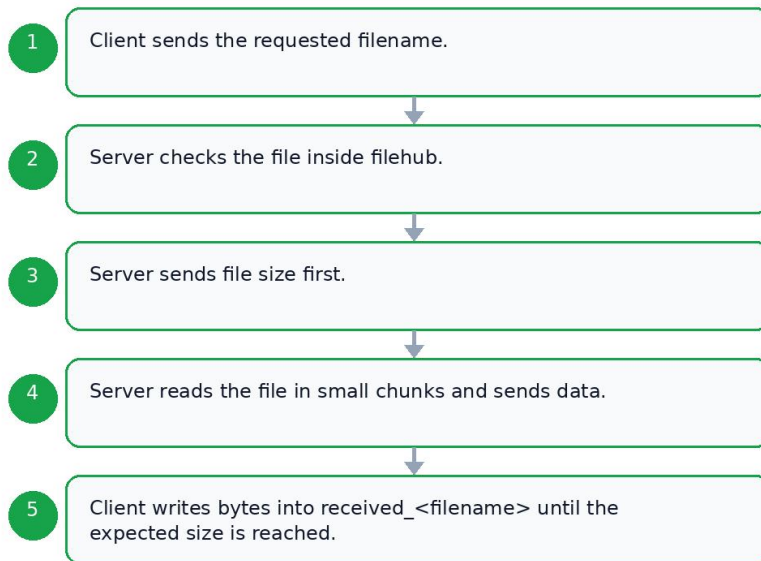


图 3.1.1 TCP 文件传输流程

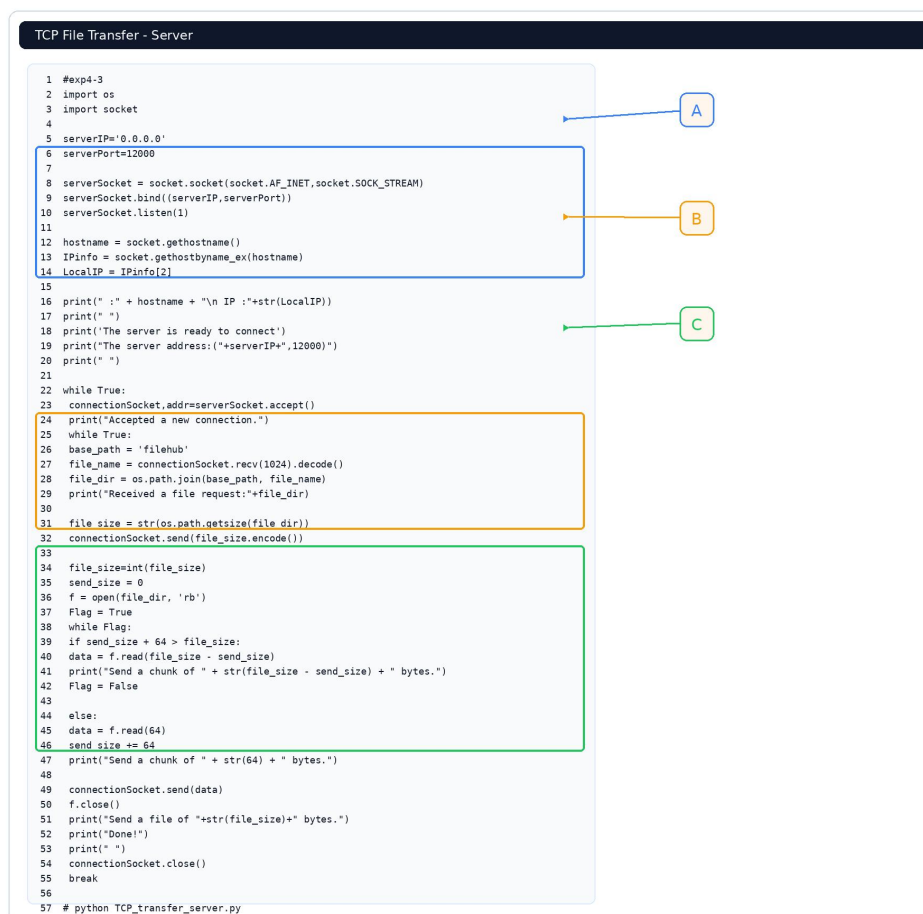


图 3.1.2 TCP 文件传输服务端核心代码

图中 A: 对应服务端初始化与监听部分。程序在此创建 TCP 套接字并等待客户端发来文件请求。

图中 B: 对应文件路径拼接与文件大小发送部分。客户端只有先拿到文件长度, 后续接收才能有明确边界。

图中 C: 对应文件分块读取与发送部分。服务端在循环中逐步发送文件内容直至结束。

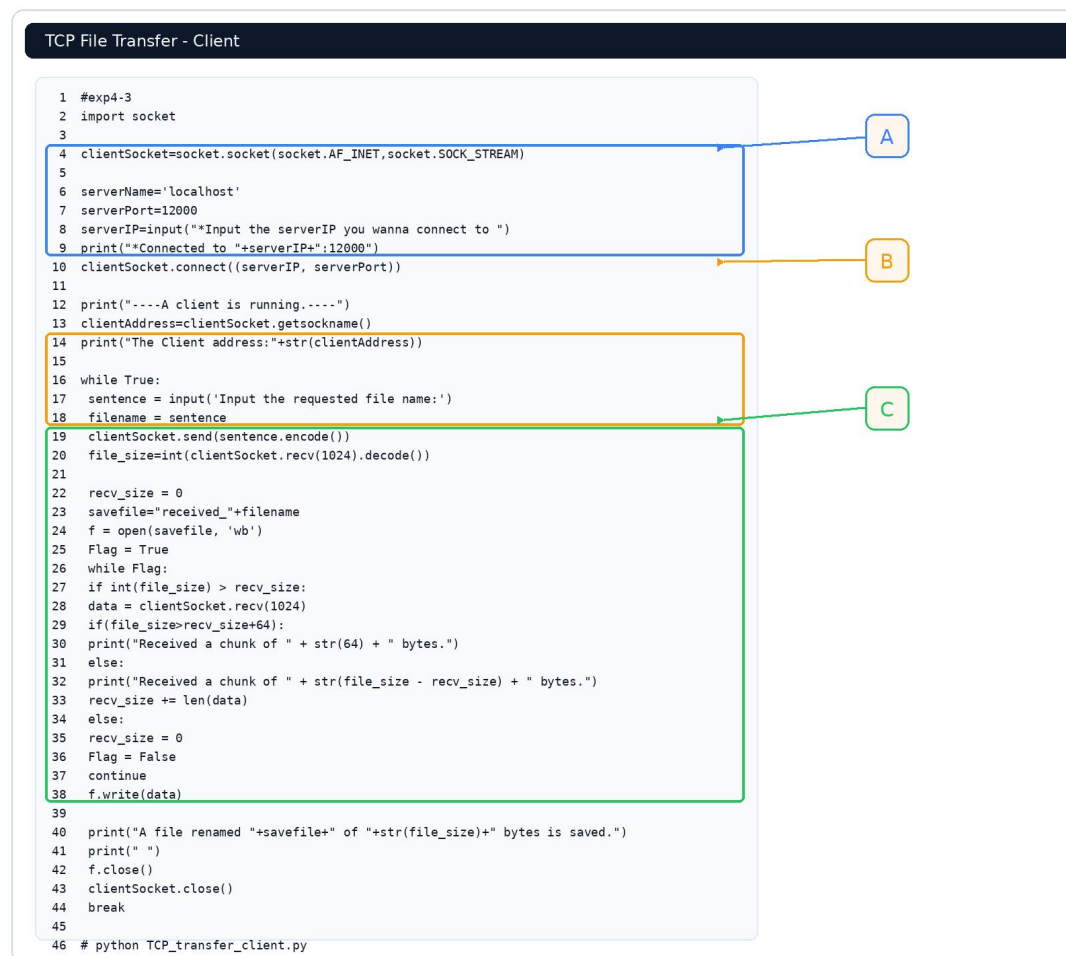


图 3.1.3 TCP 文件传输客户端核心代码

图中 A: 对应客户端连接服务器并输入请求文件名的部分。

图中 B: 对应客户端先接收文件总大小的部分。

图中 C: 对应客户端逐块写入本地文件并依据累计长度判断结束的部分。

3.2 程序运行结果

本次测试请求的文件为服务器目录中的 `short.txt`。实验结果显示, 服务端能够正确输出文件请求信息和发送过程; 客户端则能够正确显示接收块大小, 并最终生成 `received_short.txt`。

因此可以判断, 该实验已经完成了文件请求、长度传输、正文分块传输以及文件重命名保存等关键步骤。尽管测试文件体积较小, 但整个通信过程是完整且可复现的。

```
TCP File Transfer - Client and Server

# Server
Accepted a new connection.
Received a file request:filehub/short.txt
Send a chunk of 20 bytes.
Send a file of 20 bytes.
Done!

# Client
Connected to 127.0.0.1:12000
Input the requested file name: short.txt
Received a chunk of 20 bytes.
A file renamed received_short.txt of 20 bytes is saved.
```

图 3.2.1 TCP 文件传输运行结果

4 网络聊天室

要求：实现一个基于客户机/服务器的网络聊天程序，传输层使用 UDP，支持多个用户参与群聊。

4.1 代码思路与编写

聊天室部分对应脚本为 `UDP_chat_server.py` 与 `UDP_chat_client.py`。与前面几项实验相比，该部分不再是简单的一次请求与一次响应，而是需要处理多用户同时在线与消息转发的问题。

服务端的核心设计在于维护两个映射表，即 `用户名 -> 地址` 和 `地址 -> 用户名`。服务器在收到某个 UDP 数据报之后，便可据此判断该报文属于新用户加入、普通聊天消息还是退出请求。

客户端的关键在于并发处理。由于聊天程序必须同时完成“接收服务器转发消息”和“等待用户输入并发送消息”两项任务，因此程序使用 `threading.Thread` 分别启动接收线程和发送线程。

尽管 UDP 并不保证可靠传输，但其无连接、开销较小的特点，使其非常适合作为本实验聊天室的传输层协议。只要应用层对消息格式和退出规则作出明确约定，群聊功能即可顺利实现。

UDP Chat Room Logic

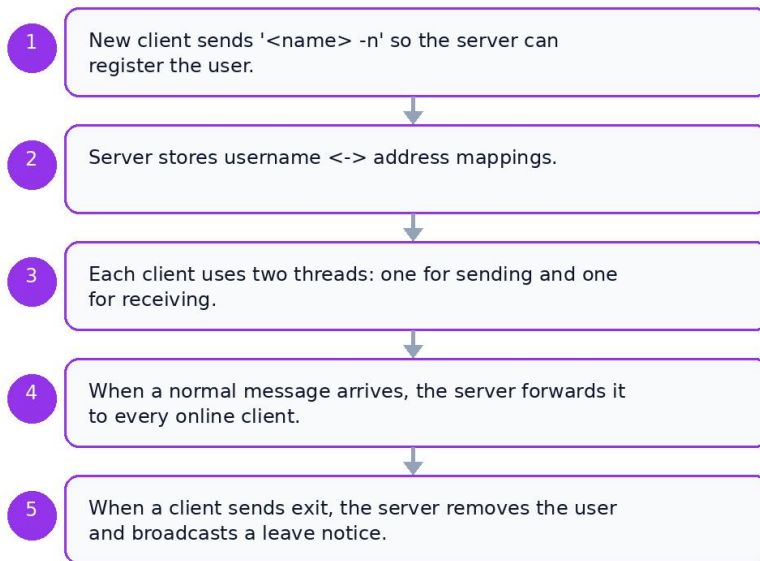


图 4. 1. 1 UDP 聊天室程序逻辑



UDP Chat Room - Client

```
1 #exp4-4
2 from socket import *
3 import time
4 from threading import Thread
5
6
7 class Client():
8     def __init__(self, name, client_post, server_post):
9
10         self.ClientPort = client_post
11         self.ServerPost = server_post
12         self.udp_socket = socket(AF_INET, SOCK_DGRAM)
13         self.userName = name
14         self.thread_rece = Thread(target=self.recv_msg)
15         self.thread_send = Thread(target=self.send_msg)
16
17     def start(self):
18         self.udp_socket.bind(self.ClientPort)
19         data_info = self.userName + ' -n'
20         self.udp_socket.sendto(data_info.encode('gb2312'), self.ServerPost)
21         print(self.getTime(), ' ')
22         self.thread_rece.start()
23         self.thread_send.start()
24         self.thread_rece.join()
25         self.thread_send.join()
26
27     def recv_msg(self):
28         while True:
29             try:
30                 rcv_data, dest_ip = self.udp_socket.recvfrom(1024)
31
32                 if rcv_data.decode("gb2312") == 'exit' and dest_ip == self.ServerPost:
33                     print(self.getTime(), ' ')
34                     break
35                 print(rcv_data.decode("gb2312"))
36             except:
37                 self.udp_socket.close()
38                 break
39
40     def send_msg(self):
41         while True:
42             data_info = input()
43             self.udp_socket.sendto(data_info.encode('gb2312'), self.ServerPost)
44             if data_info == 'exit':
45                 break
46
47     def getTime(self):
48         return '[' + time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()) + ']'
49
50     def send_message(self, message):
51         self.udp_socket.sendto(message.encode('gb2312'), self.ServerPost)
52
53 if __name__ == '__main__':
54     name = input("Please input your name:")
55     port = int(input("Please input the port:"))
56     client = Client(name, ('127.0.0.1', port), ('127.0.0.1', 12000))
57     client.start()
```

图 4.1.3 UDP 聊天室客户端核心代码

图中 A：对应客户端初始化与线程创建部分。

图中 B：对应接收线程逻辑，负责持续监听服务器转发的聊天内容。

图中 C：对应发送线程逻辑，负责读取用户输入并将消息发送至服务器。

4.2 程序运行结果

测试时同时启动两个客户端，并分别以 Alice 和 Bob 作为用户名加入聊天室。运行结果显示，Alice 发送的消息能够被 Bob 正确接收，Bob 发送的消息也能够被 Alice 正确接收；同时服务端能够完整打印消息转发过程。

由此可见，该聊天室程序已经实现了 用户加入、消息群发、用户退出 三项核

心功能，服务器维护在线用户列表的方案是有效的。

```
UDP Chat Room - Real Run Result

# Client Alice
Alice:hello from alice
Bob:chat room works

# Client Bob
Alice:hello from alice
Bob:chat room works

# Server
Receive from ('127.0.0.1', 12011) b'hello from alice'
Send to ('127.0.0.1', 12012) --> Alice:hello from alice
Receive from ('127.0.0.1', 12012) b'chat room works'
Send to ('127.0.0.1', 12011) --> Bob:chat room works
```

图 4.2.1 UDP 聊天室真实运行结果

4.3 结果补充说明

为使文档结构更接近样例报告，除真实运行结果外，本报告还补充了流程图、核心代码讲解图以及图示说明。这样处理的目的在于提高实验过程的可读性，而不仅仅是增加篇幅。

若后续仍需继续向样例报告靠拢，可进一步增加完整代码截图、本机终端界面截图，以及 GUI 聊天室版本的补充说明。

实验结果

实验结果与分析、结论如下：

1. **URL 请求程序** 已经能够完成网页下载、HTML 保存以及文件大小统计，说明基本的 HTTP 请求处理流程正确。
2. **系统时间查询** 已经按要求实现了 Time./Exit. 两类命令交互，TCP 连接的建立、响应和关闭过程清晰可见。
3. **网络文件传输** 已经完成文件名请求、文件大小发送、分块传输和客户端重命名保存等步骤。
4. **网络聊天室** 已经完成多人群聊的基本功能，客户端之间可以相互收到消息，服务端也能正确广播和维护用户列表。
5. 从整体来看，这四个实验把网络程序里最常见的几种场景都覆盖到了：**请求网页、收发指令、传输文件、转发消息**。把这些基础打牢，后面再看更复杂的网络应用就不会太吃力。

实验小结

通过本次实验可以看出，网络程序表面上是在进行消息收发，但真正决定程序是否稳定运行的，是 **消息格式、交互顺序以及结束条件** 的设计是否严谨。

前两项 TCP 实验说明，面向连接的协议在处理请求—响应式任务时具有明显优势；特别是在文件传输场景下，若缺少可靠传输和顺序保证，客户端将难以判断数据是否已经完整接收。

聊天室实验则进一步体现了 UDP 的特点。尽管 UDP 不保证可靠到达，但在轻量、即时的消息转发场景下，只要应用层协议设计合理，仍然能够取得较好的实验效果。

此外，通过参照样例报告补充流程图、代码分析图和运行截图，也能够更加清楚地认识到：实验报告不仅要说明“做了什么”，更要说明 **程序如何实现、实验现象为何出现、结果能够说明什么**。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：